CPSC4140
Fall 2026
November 9, 2026

Milestone 3: Prototype Documentation and Success Plan

RepairConnect Community Repair Kiosk

The Two-Weekers

Team 05

Rayan Ahmed, Miller Hendricks, Ford Scott, Shaun Whitt

Part 1: System Description

Problem and Purpose

RepairConnect is a community repair kiosk prototype that helps people decide what to do with a damaged item. The system brings item check-in, volunteer assessment, repair and replacement comparisons, scheduling, and ticket review into one workflow. Its purpose is to help visitors understand their options and identify a practical next step.

Milestone I identified repair-versus-replace decisions as a focused money-saving problem. People may lack information about costs, tools, skills, and available assistance. RepairConnect addresses these needs through a proposed community service in which a volunteer and visitor use the kiosk together. [1]

Users and Context

Intended visitors include college students and parents managing household expenses, as identified in Milestone I, as well as other people seeking help with damaged belongings. Volunteers are a second user group: they assess items, record findings, and log repair outcomes. The proposed kiosk would operate in a community space with access to assistance and repair resources. [1, 2]

Example User Experience

A visitor brings a backpack with a split seam to the community space. They begin a visit, select Bags & clothing and Backpack, and describe the damage. A volunteer signs in and records an assessment. Together, they compare assisted repair, professional repair, and replacement. If the visitor selects assisted repair, they choose an available session and receive a ticket displaying a sample sewing kit and locker assignment. After the repair, a volunteer records the outcome and whether the kit was returned.

Prototype Scope

The prototype is a desktop demonstration. Prices, completion times, kits, lockers, and availability are illustrative. Cases remain available only while the application is running. The prototype does not connect to a real inventory, provider directory, or appointment service. These boundaries are important when interpreting the walkthrough.

Part 2: User Tasks and Interface Walkthrough

The following walkthrough follows a backpack repair from check-in to ticket creation. Each screen capture should show the running prototype at the stated step.

Task 1: Begin the Visit and Check In an Item

1. Select “Tap to begin” on the welcome screen. Accessibility settings can be opened before starting.
2. Select Electronics, then Smartphone.
3. Enter “Broken screen with cracks on most of the screen”
4. Select Submit for review. Required-field messages explain what is missing if the category, item, or description has not been provided.

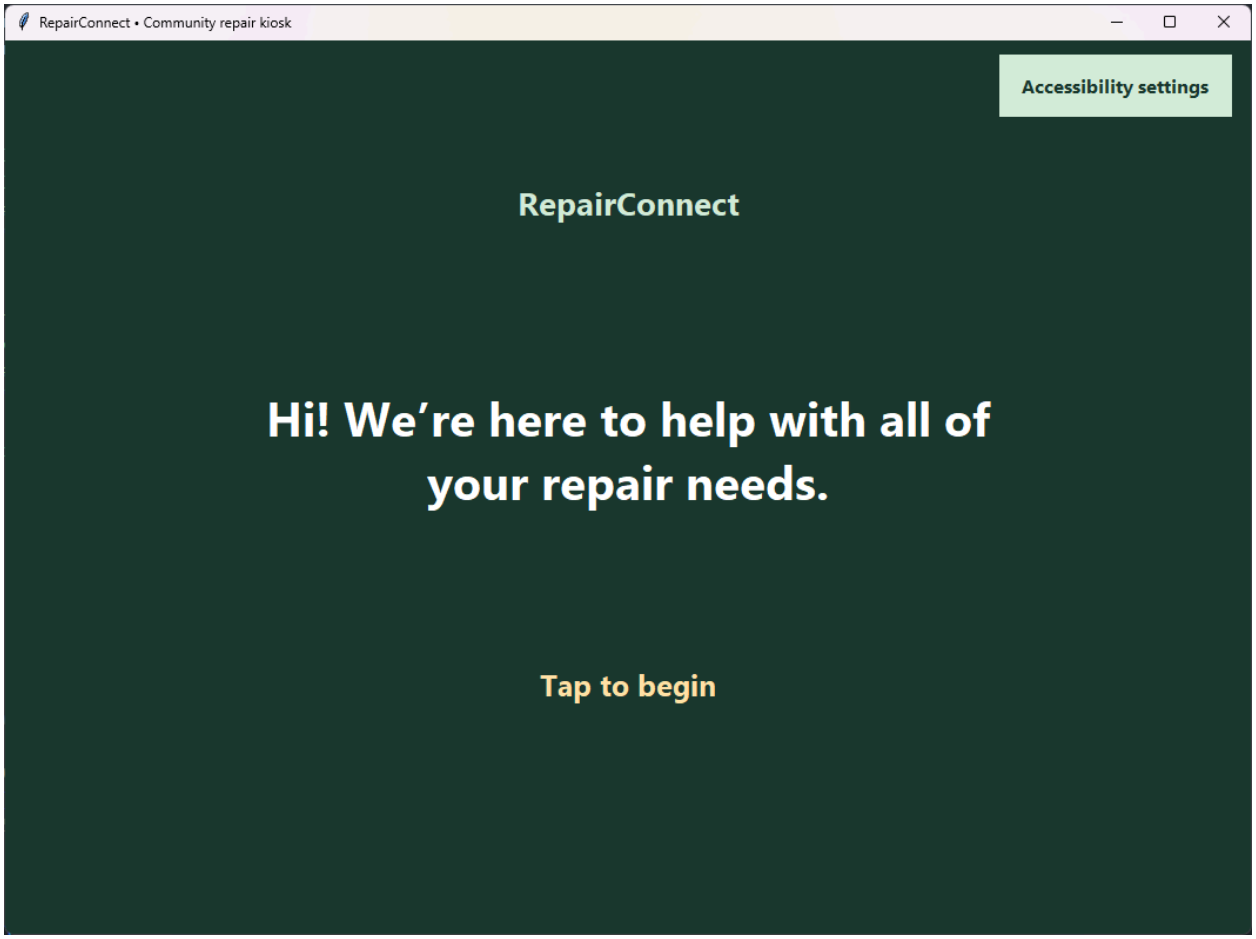


Figure 1. The welcome screen introduces RepairConnect and provides a clear starting action. The accessibility settings offer an alternative starting point for visitors who need display adjustments.

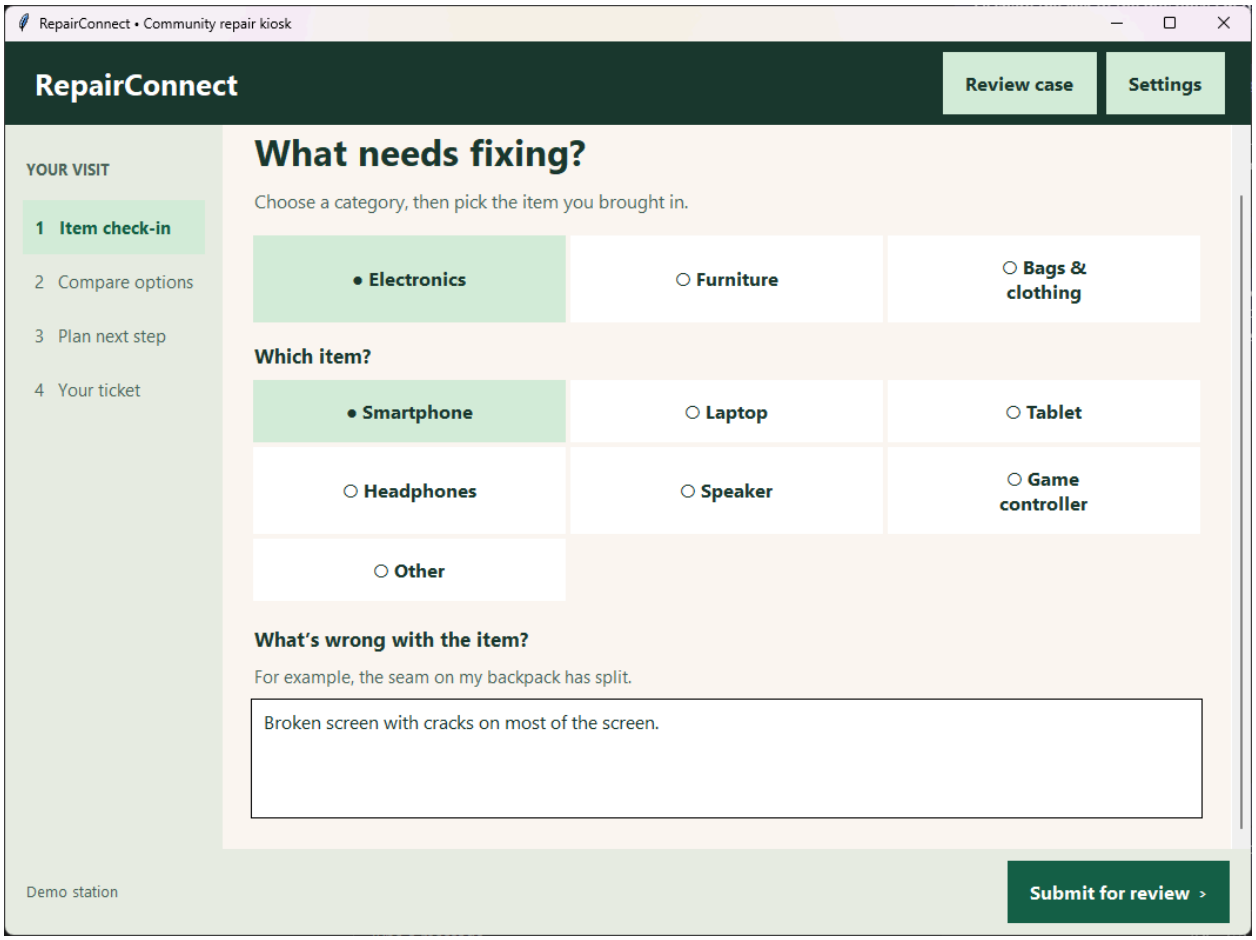


Figure 2. Category and item choices help the visitor identify their belongings, while the description field lets them explain the problem in everyday language.

Task 2: Obtain a Volunteer Assessment

1. Follow the instructions to pass the kiosk to a volunteer.
2. The volunteer selects Volunteer sign-in and enters demonstration credentials.
3. The volunteer records findings and may attach an existing image file.
4. Select Save assessment to move to the comparison screen.

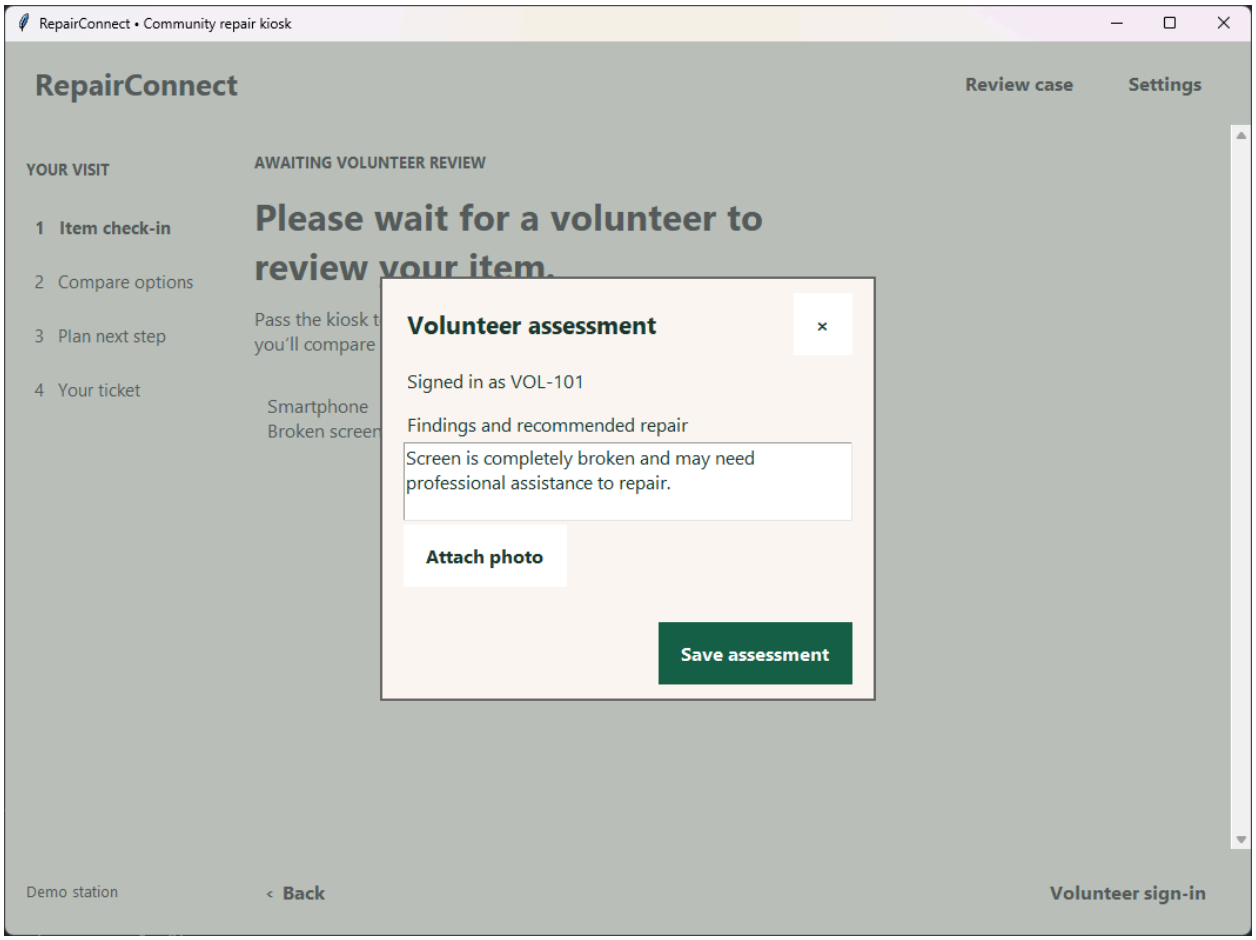


Figure 3. The handoff makes the volunteer's role explicit. The assessment dialog captures findings before the visitor chooses a path. Image attachment uses an existing file rather than a camera capture.

Task 3: Compare Options and Plan a Next Step

1. Review the assessment with the volunteer.
2. Compare Assisted repair, Professional repair, and Replacement.
3. Select a path and Continue.
4. For assisted repair, choose an available session and select Confirm reservation. A visitor can instead request a professional referral or save the case for later.

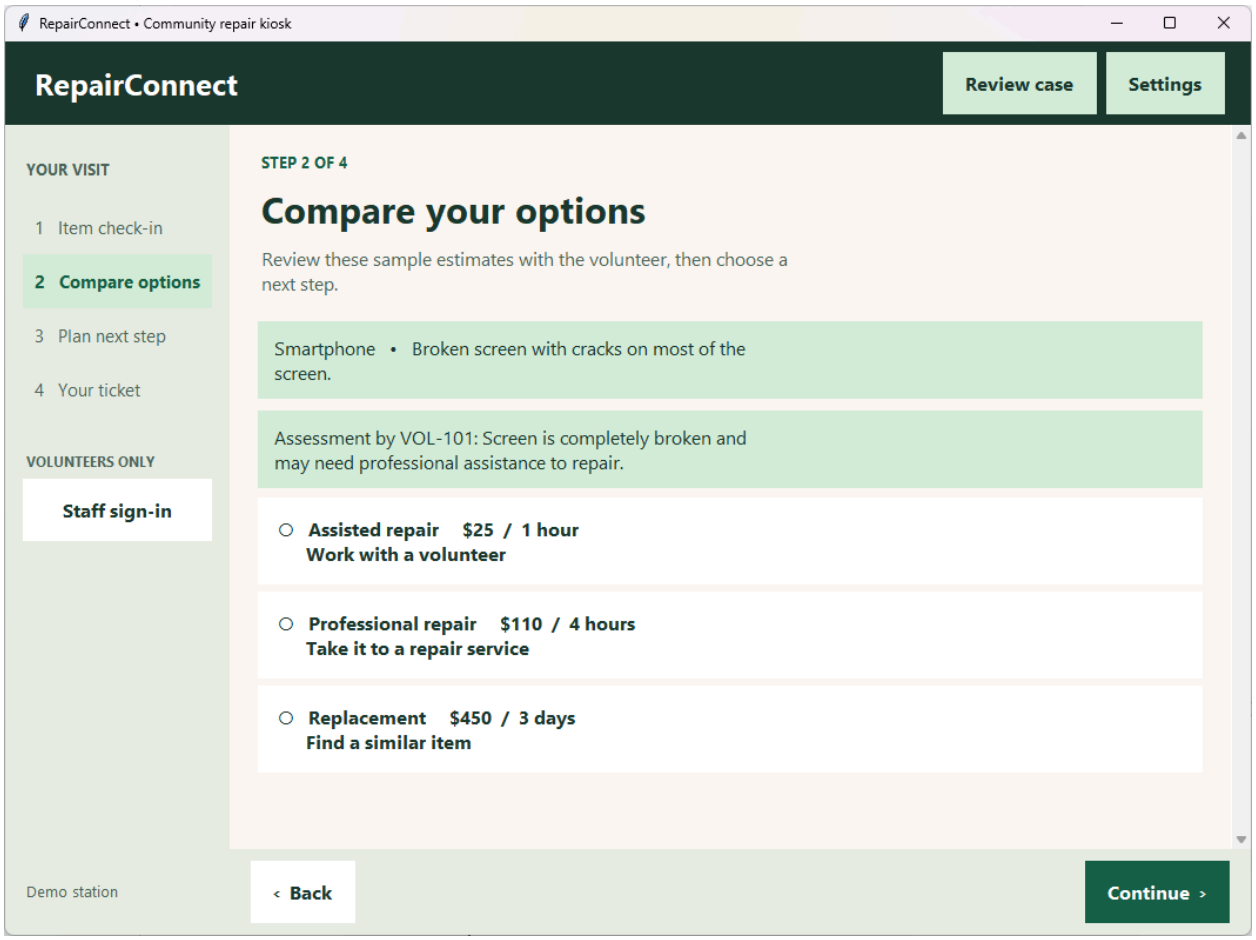


Figure 4. The comparison screen places sample prices and times together. For a smartphone, the comparison screen displays $5 for assisted repair, $110 for professional repair, and $450 for replacement. These are demonstration estimates rather than verified quotes.

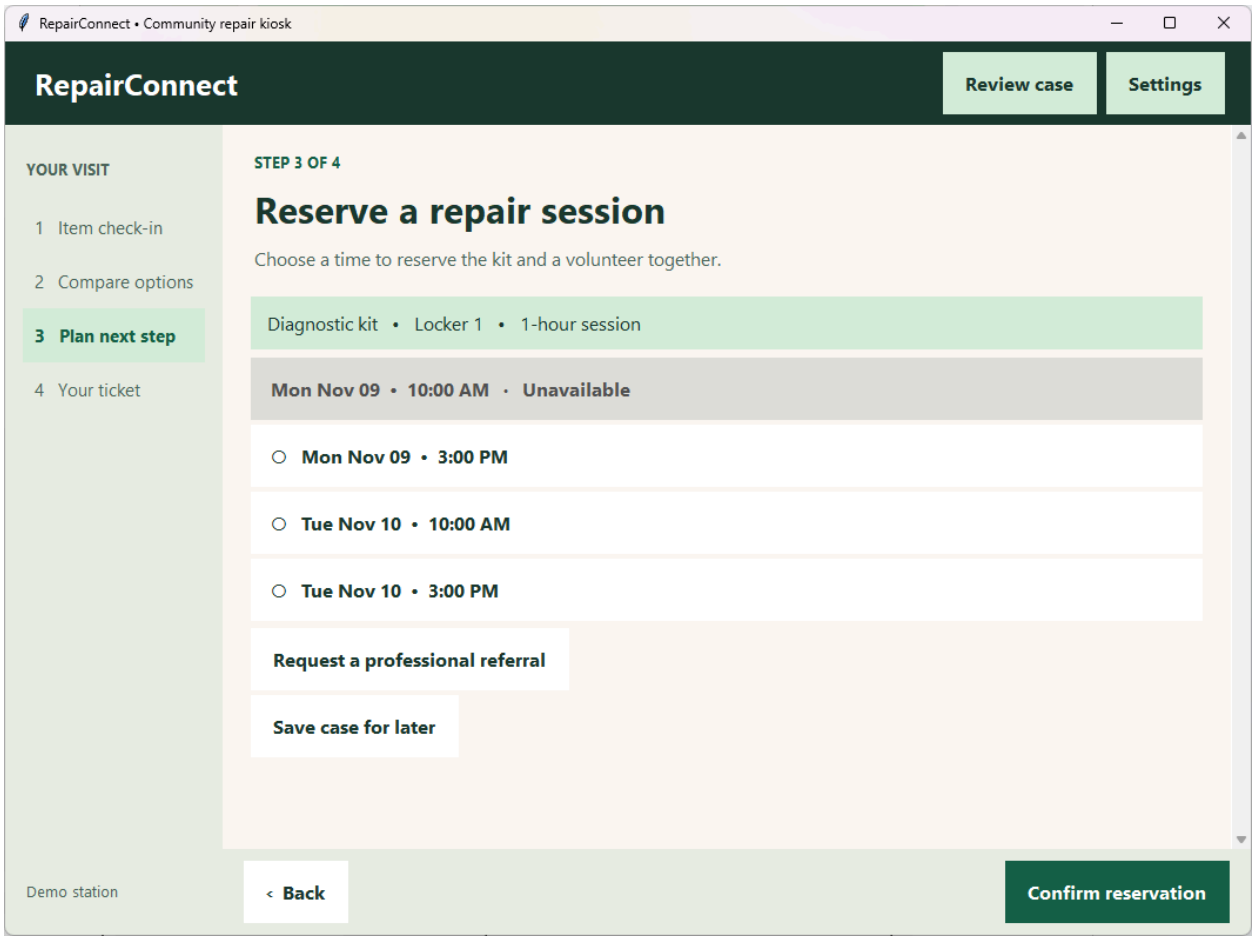


Figure 5. Session selection shows a sample repair kit, locker, and time choices. An unavailable session demonstrates a recovery situation. Professional repair and replacement instead lead to a referral or replacement-plan review; they do not book services or purchase items.

Task 4: Keep a Ticket and Review the Outcome

1. Review the ticket and retain the case code.
2. Open “Preview printed ticket” for a receipt-style preview, or “Phone ticket · QR code” for local-network sharing when available.
3. Finish the visit. During the same application session, select “Review case” and enter the code to reopen the ticket.
4. After an assisted repair, a volunteer signs in through “Log repair outcome”, selects an outcome, adds a note, and records kit return.

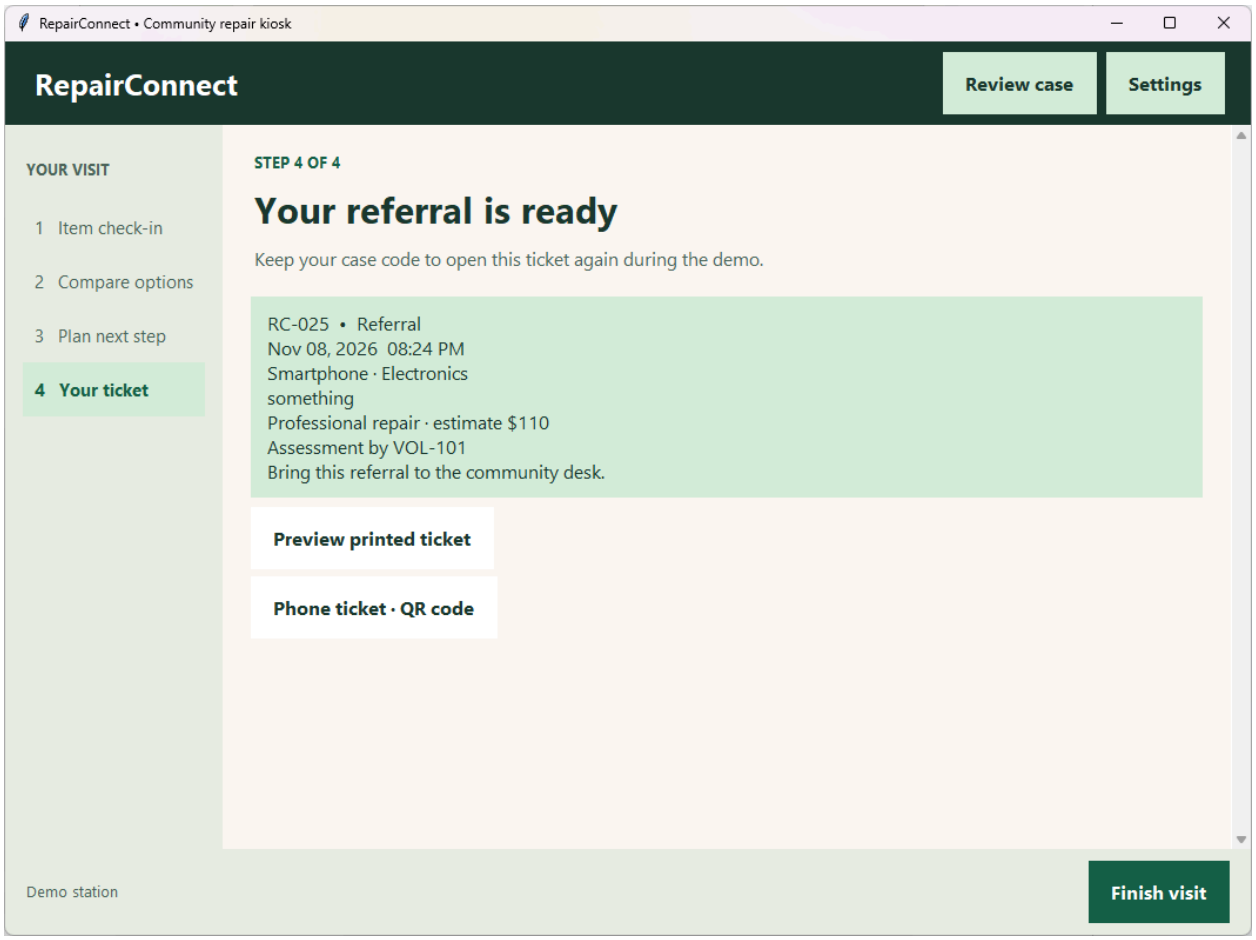


Figure 6. The ticket summarizes the chosen path and next action. A case closes when repair completion and kit return are both recorded; otherwise it remains open for follow-up. The print feature previews a receipt, and the phone link depends on the running application and a reachable local network.

Accessibility Settings

Settings include dark mode, larger text, larger buttons, and reduced welcome-screen motion. The interface also supports keyboard focus and navigation within dialogs. These features support evaluation of different interaction needs; user testing is still required to establish their effectiveness.

Part 3: Design Justification and Research Evidence

Selection of the Community Repair Kiosk

Milestone II explored a repair website, the EyeCat smart-glasses assistant, and a community repair kiosk. The kiosk provides a useful direction because it connects information with a proposed source of tools, workspace, and human assistance. Its structured sequence moves from assessment to comparison and then to a concrete plan. This rationale is consistent with the workflow implemented in RepairConnect. [2]

The design is especially suited to situations in which knowing how to repair an item is insufficient without equipment or assistance. However, its benefits depend on visitors being able to reach the location and on the community space having appropriate resources. Large or difficult-to-transport items remain a limitation. [2]

Milestone I Evidence and Design Responses

Milestone I reported 17 peer-feedback responses. Eleven classmates selected the thriftier idea as having the strongest potential, and eleven selected it as the idea they were most interested in. Peers also warned that general budgeting applications already exist, while one suggested repair-versus-replace decision support. This supported a more focused problem definition. The class sample should not be treated as representative of all intended users. [1]

The literature summarized in Milestone I linked repair feasibility to tools, expertise, money, and community support. It also described economic, environmental, personal, and social motivations for repair. The kiosk responds through volunteer review, sample cost comparisons, and a proposed shared-resource setting. A smartphone study discussed in the milestone found that cost contributed to continued use of damaged devices, reinforcing the relevance of comparing options. [1]

Milestone II Design Development

Milestone II developed the kiosk around item assessment, comparison, scheduling, tool reservation, and confirmation. RepairConnect carries these ideas into an interactive kiosk flow. The milestone also identified staffing, inventory costs, travel, opening hours, and item portability as constraints. The prototype makes the interaction reviewable but does not resolve those service-level dependencies. [2]

Studio 2 Feedback and Response

While a majority of our peers suggested we move forward with one of our other ideas, we decided to go with the second most popular recommendation. This is because the main reason people suggested we move forward with our other idea is that they thought it would be the easiest to implement which we found to not be a real issue for us. Keeping that in mind though, when we were discussing what our implementation would look like, we made sure to keep the scale of what we would have to make in mind since so many people were concerned whether or not we would be able to build out the kiosk program within the semester time period.

Studio 3 Feedback and Response

The feedback mostly described the workflow as clear, functional, and easy to follow, so we kept the same structure. When it came to concerns it mostly focused on first-time users, crowded screens, Volunteer Sign-In placement, low button contrast, description-field readability, numeric-only case lookup, and long forms. Our next revision could move Volunteer Sign-In to the top navigation, improve contrast and text,

reduce screen clutter, label optional fields, add confirmation sounds, and add case search. Staffing costs and company specific kiosk remain future potential improvements beyond this prototype.

Part 4: Implementation Challenges and Future Work

Technical and Design Limitations

The prototype stores cases in memory, so they are lost when the application stops. Scheduling uses generated time choices and demonstration availability rather than a shared calendar or inventory. Prices and times are preset. Volunteer authentication uses embedded demonstration credentials. These choices limit the prototype to demonstrating and evaluating the workflow.

Ticket delivery is also limited. The printed ticket is a preview, and phone sharing uses a local HTTP server while the application runs. Photo attachment requires the Pillow API and an existing image file. The referral and replacement paths direct the visitor to a desk rather than integrating with external providers.

Challenges Encountered and Features Not Completed

Some challenges faced when implementing the interface was to keep in mind the average user's experience with a touch screen. We wanted to make the interface as accessible as possible, so we considered every challenge that could be faced amongst the user such as small buttons or small text. These were solved using a simple accessibility setting menu that countered those challenges. Another challenge faced was implementing an on-screen keyboard, which was a thought we had when implementing the explanation side of the repair. We figured the application would call for a keyboard from the native device instead of implementing our own, which seemed far simpler than the former.

Proposed Next Steps

Future work should begin with persistent case storage and reliable retrieval, followed by real resource and appointment availability. Current, evidence-based estimates would improve the comparison. A clearer safety assessment could help volunteers identify when qualified assistance is required. Real printing or durable ticket delivery could be added if evaluation shows a need.

Evaluation with visitors and volunteers should guide these additions. Useful tasks include checking in an unfamiliar item, explaining a comparison choice, handling an unavailable session, reopening a ticket, and recording follow-up. Observations should capture completion, confusion, and participant comments before the interface is revised.

Part 5: Technology Justification

Python and Tkinter

The implementation uses Python and Tkinter to create a desktop kiosk demonstration. Tkinter supplies forms, buttons, dialogs, scrolling areas, and keyboard interaction. This supports a working sequence in which participants can enter information, encounter validation, select a path, and inspect a resulting ticket. The platform is therefore appropriate for early evaluation of the proposed interaction.

Supporting Libraries

Pillow supports opening and preparing attached images. ReportLab supplies QR-code generation, while Python's HTTP server makes explicitly shared ticket summaries accessible over a local network. These capabilities let participants evaluate photo attachment and phone-ticket access within the demonstration.

Connection to Usability and Evaluation

The four-stage progress rail helps visitors understand where they are in the visit. Plain-language labels, required-field feedback, visible selections, and back controls support task completion. Display settings allow the team to explore different viewing and interaction preferences. These features provide testable usability goals rather than proof of usability.

Evaluation should determine whether a first-time visitor understands the handoff, can compare the choices, can recover from unavailable times, and can interpret the ticket. Volunteers should also evaluate whether assessment and outcome recording fit their work. This evidence would inform both interface improvements and decisions about the next implementation platform.

Conclusion

RepairConnect demonstrates how a community kiosk could connect repair-versus-replace decision support with volunteer assistance and a planned repair session. The implementation makes the proposed workflow available for evaluation. Further development and user research are needed to connect that workflow to dependable records, realistic estimates, and a functioning community repair service.

Part 6: Initial Usability Specifications and Evaluation Plan

Task Success and Measurability Standards

In revisiting the team's early proposed tasks from milestone 1, we found that, although broad and not 100% accurate to the current product, each task is accurately accounted for and appears in the expected order the team initially proposed. Overall, to evaluate our prototype, the team plans to recruit participants to walk through the prototype interface using task scenarios while we act as the volunteer given our familiarity with the interface, which may provide a more accurate look into how real-world implementation of this interface may play out given two users (customer and volunteer), each likely with differing levels of experience with the interface. In tasks such as selecting their item and inputting its damage or evaluating repair options based on time and cost, success is largely based on the user completing the task with moderate to full understanding about the actions they are performing with minimal errors present during their execution. Each of these tasks can be evaluated based on number of clicks, time taken to complete the task, and number of errors. Likewise, the following tasks follow the workflow for evaluating a smartphone, which needs to be explicitly declared for identifying and submitting items.

Proposed Quantitative Tasks and Benchmarks

Below are two tasks for two different stakeholders, those being the average customer and the volunteer. Each of these stakeholders are directly involved in the usage of the prototype and follow distinct workflows in completing tasks. Each task is accompanied by a success metric, data collection method, and proposed benchmark

Customer - Task 1: Identify an item, describe its damage, and submit it for review.

- **Success** - Customer successfully navigates from the starting page to step 1, selects the "Electronics" and then "Smartphone" tabs, types their problem into the textbox, and presses the "Submit for review" button to reach the volunteer screen.
- **Data Collection** - Completion rate, time to complete, and incorrect selections.
- **Benchmark** - 90% of participants correctly select their item and enter their item description with no more than 1 incorrect selection in under 1 minute.

Customer - Task 2: Evaluate repair options and schedule an assisted repair for tomorrow at 3:00 P.M.

- **Success** - Customer successfully reaches the repair options screen after reading the volunteer's evaluation, reads and understands each repair option's estimated time and cost, selects "Assisted repair", chooses the time slot for tomorrow at 3:00 P.M., and presses "Confirm reservation".
- **Data Collection** - Completion rate, time to complete, and incorrect selections.
- **Benchmark** - 90% of participants independently and correctly select the correct time slot and can identify the session time and repair location, as shown on the printed ticket.

Volunteer - Task 1: Look up an existing ticket.

- **Success** - Volunteer presses the "Review case" button, ideally from the starting screen, inputs the given ticket code, and presses "Review ticket" to pull up the ticket's information.
- **Data Collection** - Completion rate, time to complete, total number of clicks, and unsuccessful searches.

- **Benchmark** - All volunteers successfully locate the “Review case” button and input the correct ticket code on the first or second try within 30 seconds.

Volunteer - Task 2: Record a short follow-up on an existing ticket and log the repair as successful in the system. Note the customer has returned their kit.

- **Success** - Volunteer presses the “Review case” button, ideally from the starting screen, inputs the given ticket code, and presses “Review ticket” to pull up the ticket’s information. From there, the volunteer presses the “Log repair outcome” button, enters their credentials, selects the “Repair complete” bubble, checks the “Diagnostic kit returned” box, types a short session note, and presses the “Close case” button.
- **Data Collection** - Completion rate, time to complete, total number of clicks, and unsuccessful searches.
- **Benchmark** - All volunteers correctly mark the kit as returned and the repair as successful with no incorrect closures in under 2 minutes.

Proposed Qualitative Data Collection Standards

Qualitative data to collect

We will collect visitor and volunteer comments about unclear labels, instructions, the visitor-to-volunteer handoff, cost and time comparisons, confidence in the selected path, and accessibility settings. Observers will also note hesitation, repeated actions, requests for help, and the reasons participants give for their choices.

Collection Methods

Visitors will think aloud while using the kiosk for a repair scenario, Volunteers will also think aloud while operating a kiosk to simulate seeing if a visitor needs help. Each session will end with a short semi-structured interview and open-ended survey questions. The same core questions will be used for each participant, and the observers will record comments and misunderstandings without coaching.

Why These Methods are Appropriate

RepairConnect is a sequential kiosk workflow shared by visitors and volunteers. Think-aloud sessions reveal confusion at the moment it occurs, while interviews explain confidence, trust, and decision reasoning. Open-ended survey responses provide consistent feedback across participants. Together, these methods reveal usability problems that completion times and error counts alone cannot provide.

Part 7: AI Usage and Documentation

Initial Prompt for Prototype and Subsequent Revisions

Given the time constraints and limited team knowledge regarding the prototype's used Python dependencies, including QR code functionality, functional screens and login services, temporarily saved data, and general interface programming, the team opted to utilize AI code generation for the prototype's initial version and subsequent revisions for quicker development and receptive change iterations. The team used ChatGPT's Astra model for all responses. The below prompt is what the team used to receive the first iteration of the RepairConnect interface:

"Good afternoon! I am wanting to create a prototype interface for a repair system kiosk, as shown and described in the section titled "Idea 3: Community Repair Scheduler (Kiosk)" of the attached document [EDITOR'S NOTE: The document in question was the team's milestone 2 PDF. This was done so Astra knew what the team was looking for in terms of functionality and general design].

The design ideas of the repair kiosk are written in the document. The interface should follow a reasonable workflow and adhere by design principles of human-computer interaction. The script should be written in Python and import all necessary libraries.

If you require further information on what the interface should contain, please ask before beginning. Remember that this is a system prototype, NOT a finished product, so the interface does not to be fully complete. However, functionality, like writing in text boxes, pressing buttons like for category selection, opening menus like settings, and feedback loops when moving to the next section, should all be clearly presented and working. Make sure there's color too! Everyone likes a friendly interface."

After the first prototype was received, the foundation of the user workflow and system interface were set. However, the first version featured several critical errors and logical mistakes, including settings and volunteer views opening in a new window (the team's limitations in access to a kiosk or other similar device were a direct cause of this), ambiguous buttons that may accidentally be pressed by a user without knowing what they do, lack of common, pre-configured items for users to choose from when selecting an item category, volunteer/credential association with tickets for logging purposes, unclear time slot unavailability and immediate error messages associated with it, and lack of accounting for electronic devices for tickets. After prompting Astra to account for these changes, another iteration was produced that was significantly better than the previous product, but still contained a few flaws, including no clear transition between power from the user to the volunteer, lack of accessibility features, and lack of functionality of recalling/closing a ticket when a repair is complete, regardless of success. After prompting Astra for these changes, the final prototype used in the studio session and analysis of this paper was produced.

All shortcomings or errors in the prototype's design were noticed and analyzed exclusively by the team with no AI assistance. AI was used exclusively for implementing suggestions the team had for future revisions to make the final product as simple to use and HCI-compliant as possible.

Makefile and readme.txt

As the proposed interface is designed for touchscreen devices (given it designed to be kiosk-oriented), compilation guided by a Makefile or remote execution for a Linux server were not accounted for in the team's technical analysis. Given the team's unfamiliarity with such a Python program in Makefile and external server compilation/execution, the team felt it was adequate to inquire Astra within the same chat

context as the prototype to garner if such documentation was possible, given the program's used dependencies. Astra responded that the Makefile could be produced like any other file, done by defining the Python version (chose python3 for maximum compatibility), file name, and compile and run directives. As a safeguard, Astra suggested including a .PHONY rule, which ensures when the Makefile is run, both the compile and run directives can execute successfully even if a file sharing the same name as either directive is present in the same working directory as the file on the Linux system. In the same convention as selecting python3, the team chose to include it for maximum compatibility. However, running the file on a School of Computing Linux computer would prove to be much more difficult, if not impossible. As running a file on the School of Computing server is dependent on the authorized dependencies defined by the administrators, it wouldn't be known if the team's interface would function correctly until after testing. The team asked Astra how to install the dependencies on a remote server, and after attempting to install the dependencies through a virtual environment, the team determined it wasn't possible due to a "no display name" error, which not only requires an administrator to install xauth, but also requires the user to to have access to their local computer AND the ssh server, which is not permitted in the School of Computing and proposed security vulnerabilities. Therefore, in its current state, the team's program can not be run on the School of Computing computers. However, Astra's provided readme.txt is included in the submission of the milestone with a disclaimer at the top.

References

[1] The Two-Weekers, Team 05. Milestone I: Identifying the Problem. CPSC4140, September 16, 2026.
[2] The Two-Weekers, Team 05. Milestone II: Drastically Different Designs. CPSC4140, October 5, 2026.